

Entrega 1

Sistemas Operacionais II

ARTHUR ERPEN

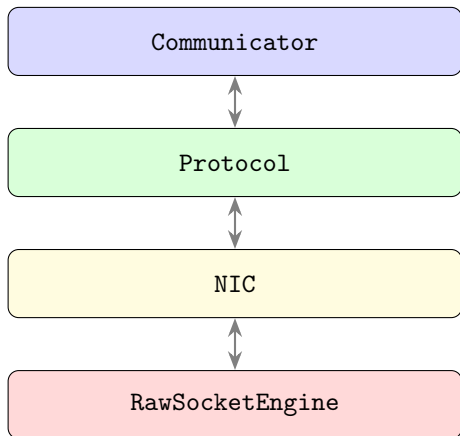
CAETANO PERUZZO

GABRIEL HERRERA

UFSC – INE5424

2026

Hierarquia do Projeto



- Camada mais baixa da pilha
- Interage diretamente com o socket raw (AF_PACKET)
- Descobre MAC local via `ioctl(SIOCGIFHWADDR)`



RawSocketEngine

□ `_sockfd : int = -1`

□ `_ifindex : int = 0`

◆ `RawSocketEngine()`

◆ `engine_init(interface_name : const char*) : void`

◆ `engine_send(frame : const void*, size : unsigned int) : int`

◆ `engine_receive(frame : void*, size : unsigned int) : int`

◆ `engine_close() : void`

◆ `engine_get_address(mac : unsigned char*) : void`

- Gerencia pool de buffers (free-list com `std::stack + mutex`)
- 8 buffers de envio + 16 de recepção (Traits)
- Aloca buffer com Frame Ethernet preenchido (MAC src/dst + EtherType)
- Utiliza a Engine para enviar os bytes do buffer
- Demultiplexação baseada no EtherType (Conditional_Data_Observed)

```

NIC<Engine>
o BUFFER_SIZE : unsigned int

□ _address : Ethernet::Address
□ _running : std::atomic<bool>
□ _thread_started : std::atomic<bool>
□ _recv_thread : pthread_t
□ _buf_mtx : std::mutex
□ _statistics : Statistics
□ _buffer : Buffer<Ethernet::Frame>{BUFFER_SIZE}
□ _free_list : std::stack<unsigned int>

● NIC()
● ~NIC()
● send(dst : Ethernet::Address, prot : Ethernet::Protocol, data : const void*, size : unsigned int) : int
● receive(src : Ethernet::Address*, prot : Ethernet::Protocol*, data : void*, size : unsigned int) : int
● alloc(dst : Ethernet::Address, prot : Ethernet::Protocol, size : unsigned int) : Buffer<Ethernet::Frame>*
● send(buf : Buffer<Ethernet::Frame>*) : int
● free(buf : Buffer<Ethernet::Frame>*) : void
● unmarshal(buf : Buffer<Ethernet::Frame>*, src : Ethernet::Address*, dst : Ethernet::Address*, data : void*, size : unsigned int) : int
● address() : const Ethernet::Address&
● address(address : Ethernet::Address) : void
● statistics() : const Statistics&
● attach(obs : Conditional_Data_Observer<Buffer<Ethernet::Frame>, Ethernet::Protocol>*, prot : Ethernet::Protocol) : void
● detach(obs : Conditional_Data_Observer<Buffer<Ethernet::Frame>, Ethernet::Protocol>*, prot : Ethernet::Protocol) : void

■ alloc_buf() : Buffer<Ethernet::Frame>*
■ recv_loop_entry(arg : void*) : void*
■ recv_loop() : void

```

Protocol

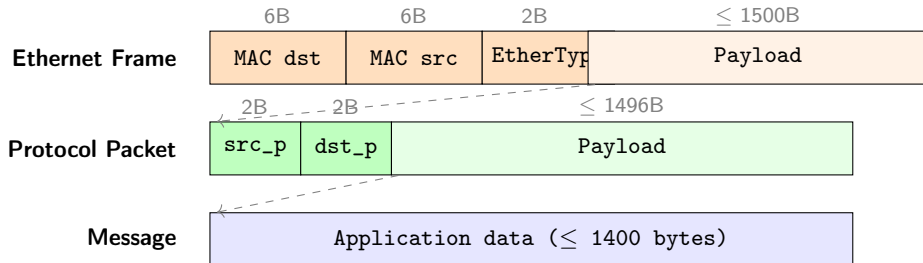
- Define pacote com cabeçalho:
src_port + dst_port
- EtherType customizado: 0x8888
- Singleton estático (_instance)
- Observer de NIC
(Conditional_Data_Observer)
- Observed para Communicators
(Concurrent_Observed)
- Demultiplexação de pacotes por porta
de destino

Protocol<NIC>	
○	PROTO : NIC::Protocol_Number
○	MTU : unsigned int
□	_nic : NIC*
□	_observed : Concurrent_Observed<Buffer<Ethernet::Frame>, uint16_t>
□	_instance : Protocol<NIC>*
◆	Protocol(nic : NIC*)
●	Protocol()
●	send(from : Address, to : Address, data : const void*, size : unsigned int) : int
●	receive(buf : Buffer<Ethernet::Frame>*, from : Address*, data : void*, size : unsigned int) : int
●	create_address(port : uint16_t) : Address
●	attach(obs : Concurrent_Observed<Buffer<Ethernet::Frame>, uint16_t>*, address : Address) : void
●	detach(obs : Concurrent_Observed<Buffer<Ethernet::Frame>, uint16_t>*, address : Address) : void
■	update(prot : NIC::Protocol_Number, buf : Buffer<Ethernet::Frame>*) : void

- API para a aplicação: `send()` e `receive()`
- Opera sobre a classe `Message` (payload $\leq$ 1400 bytes)
- Herda de `Concurrent_Observer`
- `receive()` é bloqueante — aguarda semáforo
- Abstraído pela interface `Communication_Endpoint`

C Communicator<Channel>	
□	<code>_channel : Channel*</code>
□	<code>_address : Address</code>
●	<code>Communicator(channel : Channel*, address : Address)</code>
●	<code>Communicator()</code>
●	<code>send(message : const Message*) : bool</code>
●	<code>receive(message : Message*) : bool</code>
■	<code>update(c : Channel::Observer::Observing_Condition, buf : Buffer*) : void</code>

Formato do Frame



NIC → Protocol

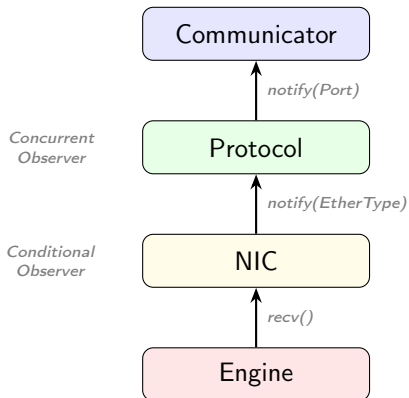
Conditional_Data_Observer

- Condição: EtherType
- Síncrono (mesmo thread)
- NIC recebe frame → notifica Protocol que observa 0x8888

Protocol → Communicator

Concurrent_Observer

- Condição: Port
- Assíncrono (semáforo POSIX)
- Protocol extrai porta → notifica Communicator correspondente



Communication Endpoint

- interface de comunicação usada pelo Component
- a classe Channel_Endpoint herda de Communication_Endpoint conectando um Channel genérico a classe communicator
- isso permite que um Component use um Communication_Endpoint qualquer

Communication Endpoint

```
class Communication_Endpoint {  
public:  
    virtual ~Communication_Endpoint() {}  
  
    virtual bool send(const Message *msg);  
    virtual bool receive(Message *msg);  
};
```

Channel Endpoint

```
template <typename Channel>  
class Channel_Endpoint : public  
    Communication_Endpoint {  
public:  
    // ... overrides methods  
  
private:  
    Communicator<Channel> _communicator;  
};
```

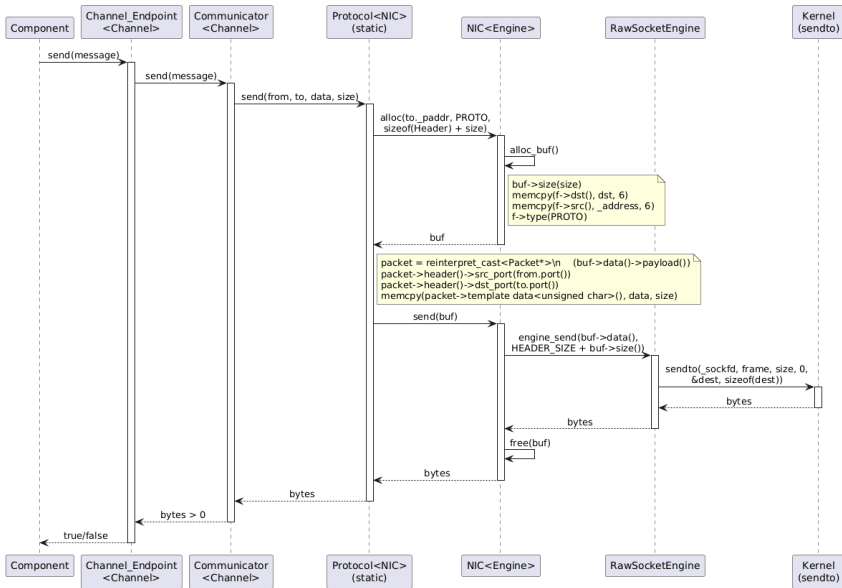
Instanciação dos Componentes

- `Vehicle::run()` faz `fork()` para cada componente
- Processo filho cria pilha própria: `NIC` → `Protocol` → `Endpoint`
- Componente recebe endpoint via injeção de dependência
- Pai aguarda filhos com `waitpid()`
- `_exit(0)` evita destrutores do pai

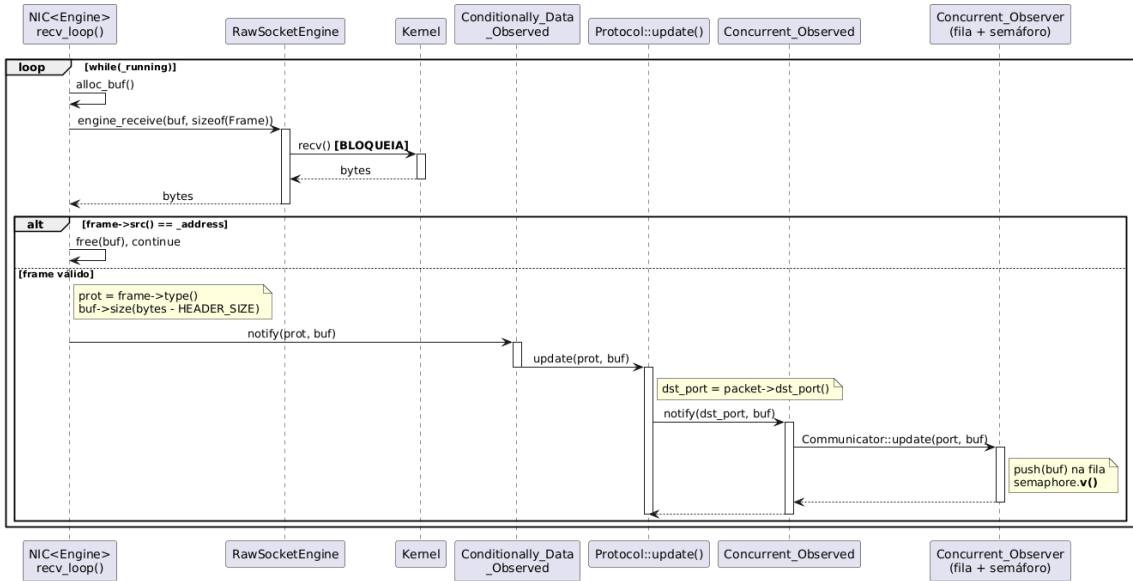
Vehicle::run()

```
for (auto c : _components) {  
    pid_t pid = fork();  
    if (pid == 0) {  
        NIC<RawSocketEngine> nic;  
        Vehicle_Protocol prot(&nic);  
        Channel_Endpoint<Vehicle_Protocol>  
            ep(&prot,  
              prot.create_address(  
                  c.second));  
        c.first->set_endpoint(&ep);  
        c.first->run();  
        _exit(0);  
    }  
    pids.push_back(pid);  
}  
// waitpid() para cada filho
```

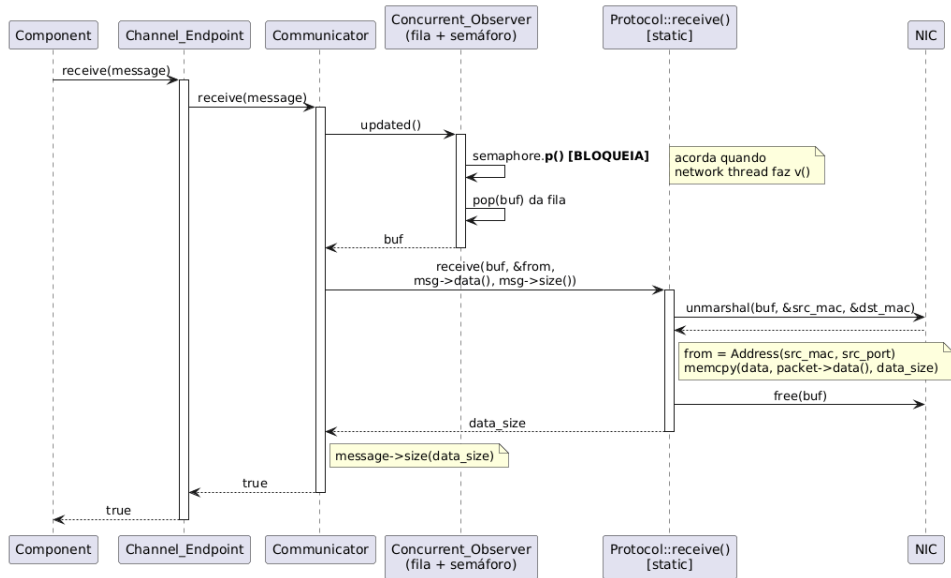
Fluxo de Envio — Send Path



Recepção — Network Thread (NIC::recv_loop)



Recepção — Application Thread



Teste: Mesh Concorrente

Cenário:

- 5 VMs QEMU RISC-V 64-bit
- Todas enviam e recebem simultaneamente
- $2 \text{ rounds} \times 3 \text{ mensagens} = 6 \text{ envios/VM}$
- $4 \text{ VMs} \times 6 = 24 \text{ recebimentos/VM}$

Validações:

- Sem duplicatas
- Sem mensagens próprias
- Metadados corretos (round, sender, seq)
- Contagem exata de envios e recebimentos

Payload

```
// "mesh-concurrent:r1:vm3:m2"  
snprintf(buf, size,  
         "%s:r%d:vm%d:m%d",  
         label, round,  
         sender_id, seq);
```

Execução

```
$ make test-mesh-concurrent  
# 5 QEMUs lançados  
# Cada VM reporta:  
# "concluido com 6 envios e  
# 24 recebimentos validados."
```

Benchmark: RTT Ping-Pong

Por que RTT e não pcap?

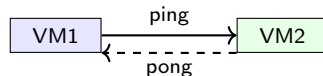
- Pcap por VM: cada QEMU é um processo separado, timestamps não são comparáveis entre VMs
- RTT: dois timestamps do **mesmo clock** (VM1)
- Zero problemas de sincronização

Arquitetura do teste:

- VM1 envia ping → VM2 responde pong
- $RTT = t_{end} - t_{start}$ (CLOCK_MONOTONIC_RAW)
- VM3–VM5: tráfego de fundo
- 20 amostras × 10 repetições
- Latência $\approx RTT/2$

Medição (VM1)

```
for (seq = 1; ...; ++seq) {  
    t1 = monotonic_raw_us();  
    send("ping", seq);  
    receive_expected(  
        "pong", VM2, seq);  
    t2 = monotonic_raw_us();  
    samples.push(t2 - t1);  
}
```



RTT médio:
6.36 ms

Menor RTT:
2.58 ms

Valores obtidos após 10 execuções
(200 amostras totais)

Como executar

```
$ make measure-rtt-10
# 10 execucoes automaticas
# 20 amostras RTT por execucao
# Script Python agrega resultados
```

Script de analise

```
# summarize_rtt.py
# Extrai RTT_SAMPLE do log da VM1
# Calcula media, min, max
# Estima latencia = RTT/2
```


Atendimento aos Requisitos

- + **C++ com bibliotecas padrão**
Sem dependências externas
- + **API minimalista**
Communicator → Protocol → NIC → Engine
- + **VM QEMU como sistema autônomo**
RISC-V 64-bit, compilação estática
- + **Componente como processo POSIX**
fork() para cada componente
- + **Raw sockets**
AF_PACKET + SOCK_RAW
- + **Broadcast em rede virtual**
MAC FF:FF:FF:FF:FF:FF via multicast
- + **Observer + eventos assíncronos**
Thread de rede → semáforo → aplicação
- + **Cenário de teste**
5 VMs, malha completa, validação automática

Entrega 1

Sistemas Operacionais II

ARTHUR ERPEN
CAETANO PERUZZO
GABRIEL HERRERA